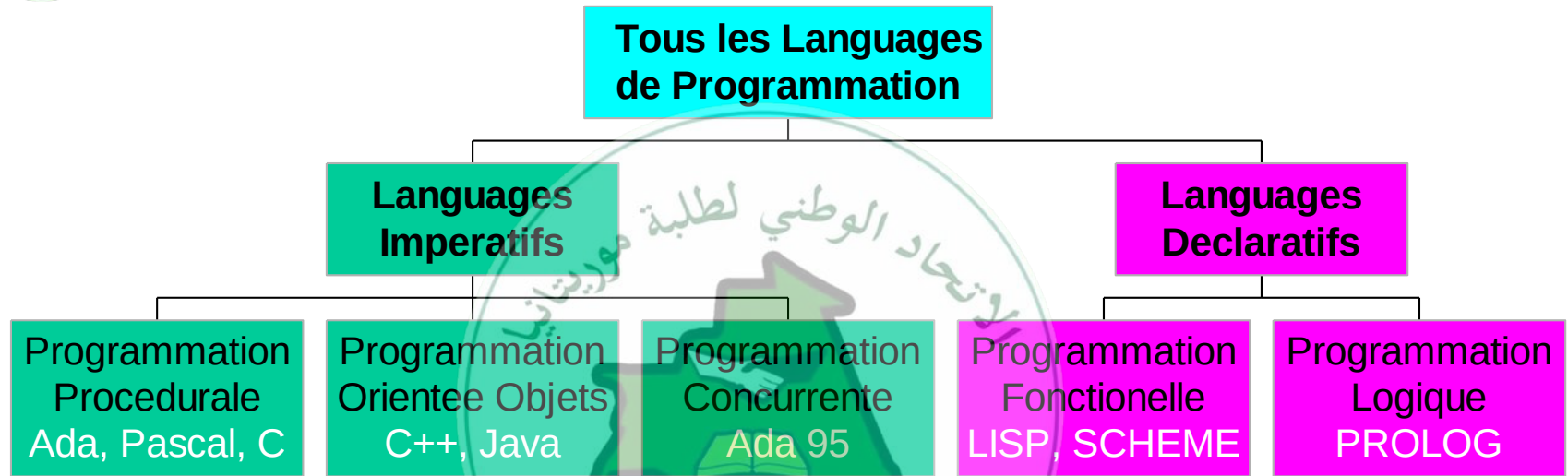




la programmation en langage C - Introduction



Languages Imperatifs: Languages incluant des moyens pour le programmeur d'attribuer des valeurs a des locations en mémoire.

Languages Declaratifs: Languages pour lesquels le programmeur réfléchit en terme de valeurs des fonctions et de relations entre entités diverses. Il n'y a pas d'attribution de valeurs aux variables.

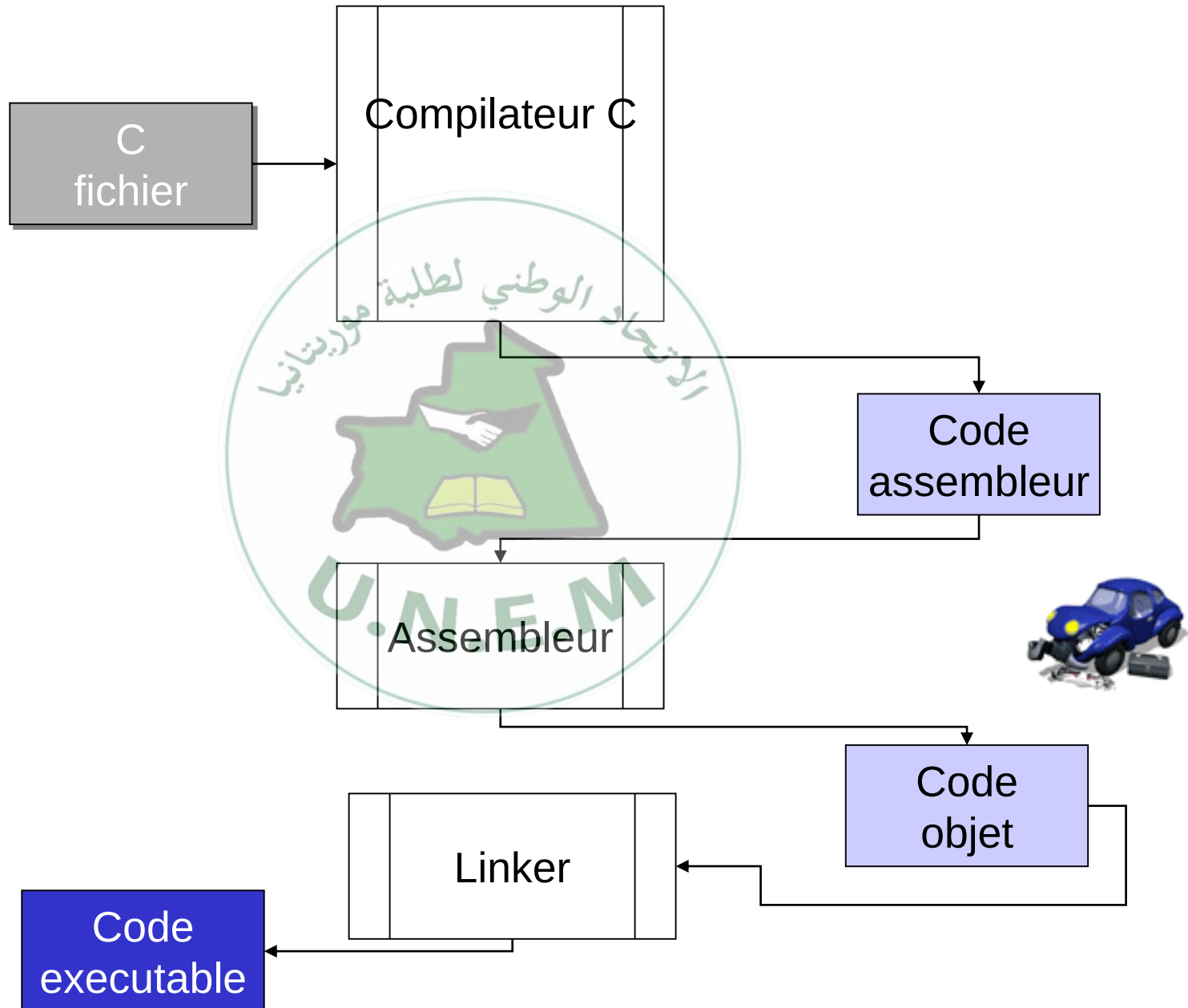
Caractéristique du C



- **Modulaire**: peut être découpé en modules qui peuvent être compilés séparément
- **Universel**: n'est pas orienté vers un domaine d'application particulier
- **Typé**: tout objet C doit être déclaré avant d'être utilisé
- **Portable**: sur n'importe quel système en possession d'un compilateur C



Un long fleuve tranquille



Fichier C (extension .c)



/* exemple de programme C :

- somme des nb de 1 à 10 et affichage
de la valeur*/

```
#include <stdio.h>
int main (void)
{
    int somme; int i;
    somme = 0;
    for (i = 1; i <= 10; i++)
    {
        somme = somme + i;
    }
    printf ("%d\n", somme);
    somme = 0;
}
```

En C le programme principal
s'appelle toujours *main*

déclarations de variables de
type entier (cases mémoire
pouvant contenir un entier)

instruction d'affectation de
valeur à la variable *somme*

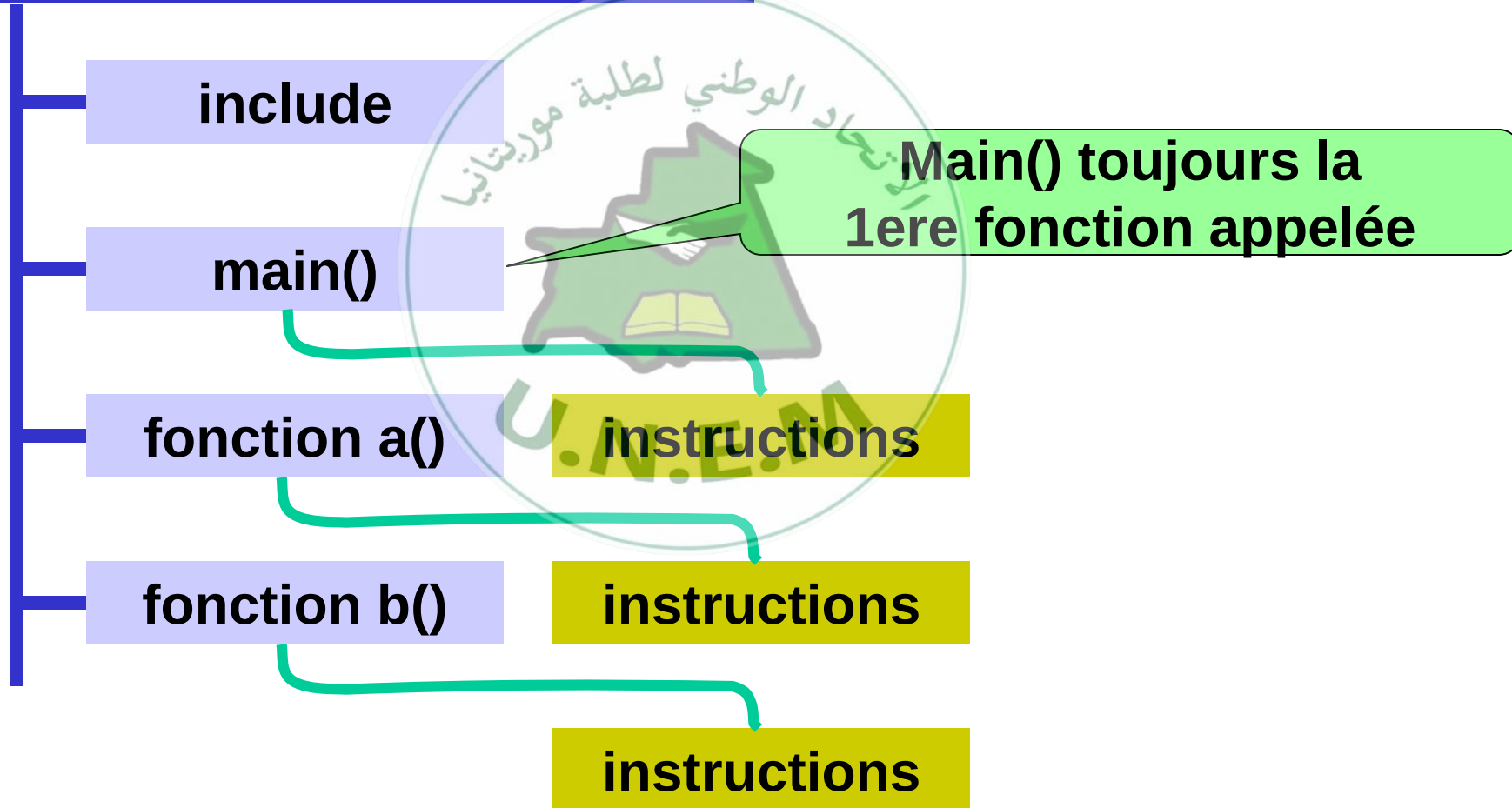
instructions exécutées
en séquence

l'instruction entre accolades
est exécutée pour les valeurs
de *i* allant de 1 à 10

affiche à l'écran la valeur de
l'entier contenu dans *somme*



Programme typique en C



Types d'instruction en C



- Déclarations des variables

- Assignations

- Fonctions

- Contrôle



Types de données et de variables



■ Déclaration des variables

- `int y;`
- `char yesno, ok;`
- `int ordered = 1, onhand = 0;`
- `float total = 43.132;`
- `char *cptr = NULL;`



Types de données et de variables



■ « char » ou « signed char »:

- ASCII sur 32 bits
 - de -2147483648 à 2147483647.

■ Type « unsigned char »:

- ASCII sur 32 bits
 - de 0 à 4294967295.



Types de données et de variables



- « short » ou « signed short »
 - en complément à 2 sur 32 bits
 - de -2147483648 à 2147483647.

- Type « unsigned short »:
 - binaire sur 32 bits
 - de 0 à 4294967295.



Types de données et de variables



- « int » ou « signed int »
 - en complément à 2 sur 32 bits
 - de -2147483648 à 2147483647.

- Type « unsigned int »:
 - binaire sur 32 bits
 - de 0 à 4294967295.



Types de données et de variables



- Type « long » ou « signed long »
 - en complément à 2 sur 32 bits
 - de -2147483648 à 2147483647.

- Type « unsigned long »:
 - binaire sur 32 bits
 - de 0 à 4294967295.



Types de données et de variables

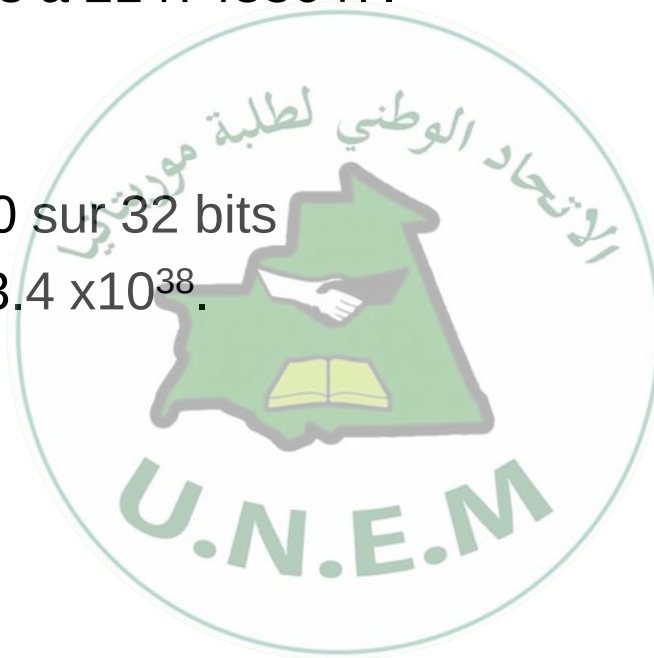


■ Type « enum » (ordered list)

- en complément à 2 sur 32 bits
 - de -2147483648 à 2147483647.

■ Type « float »

- format TMS320C30 sur 32 bits
 - de 5.9×10^{-39} à 3.4×10^{38} .



Types de données et de variables

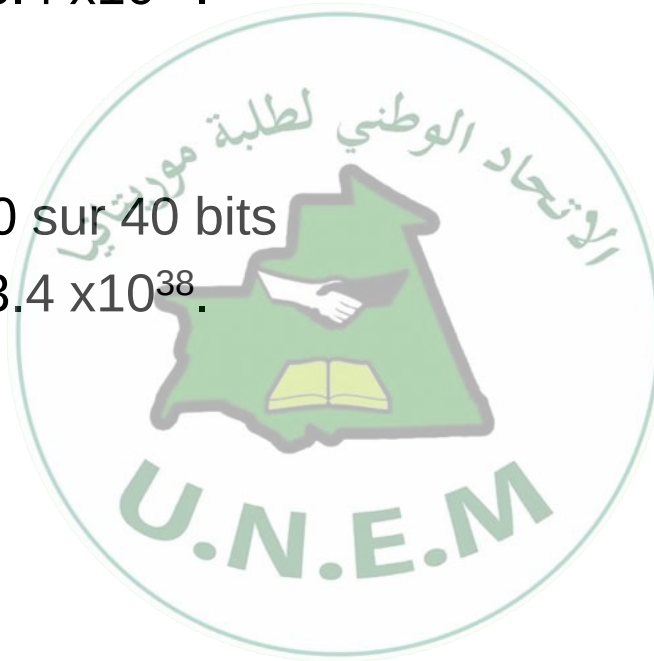


double »

- format TMS320C30 sur 32 bits
 - de 5.9×10^{-39} à 3.4×10^{38} .

Type « long double »

- format TMS320C30 sur 40 bits
 - de 5.9×10^{-39} à 3.4×10^{38} .



Types de données et de variables



pointer »

- binaire sur 32 bits
 - de 0 à 0xFFFFFFFF.



Les modificateurs des classes de mémorisation



»:

➤ indique une variable ou une fonction déclarée dans un autre module.

« register »:

➤ demande au compilateur de placer des variables dans les registres du CPU. Augmente la vitesse de traitement.



Les modificateurs des classes de mémorisation



■ « const »:

- indique que le contenu d'une variable ne doit pas être modifiée.

■ « volatile »:

- indique qu'une variable peut voir son contenu changer à tout moment par le programme, des interruptions ou tout autre facteur extérieur. Empêche le compilateur de faire des optimisations sur cette variable.

Opérateurs et expressions



■ Opérateurs à un paramètre:

- - change le signe de la variable
- &adresse
- ++/-- incrémentation/décrémentation
- sizeof()



Opérateurs et expressions



■ Opérateurs arithmétique:

■ $*, /, +, -$

■ $\%$ modulo

■ Opérateurs sur bits:

■ $<<, >>$ décalage à gauche ou à droite

- `status = byte << 4;`

■ $\&$ et

■ $|$ ou

■ $\wedge$ ou exclusif



Opérateurs et expressions



- Opérateurs relationnels:

- $<, >, <=, = >$

- Opérateurs d'égalité:

- $==, !=$

- Opérateurs logiques:

- $\&\&$

- $||$

et
ou



Opérateurs et expressions



■ Opérateur conditionnel:

- `result = mode > 0 ? 1 : 0;`
- `if mode>0 then result=1 else result=0.`

■ Opérateurs d'assignation:

- `=, *=, /=, %=, +=, -=, <<=, >>=, &=, |=, ^=`



Fonctions en C (Exemple)



- Les fonctions pré-définies:

- `printf()`, `sin()`, ...

- Le prototype de ces fonctions sont dans fichiers d'entête (header file)

- `printf()` dans `stdio.h`

- `sin()` dans `math.h`



Fonctions en C (Exemple)



- Par exemple, nous pouvons écrire nos propres fonctions.

```
/* Routine de calcul du maximum */  
int imax(int n, int m)  
{  
    int max;  
  
    if (n>m)  
        max = n;  
    else  
        max = m;  
    return max;  
}
```

Déclaration de la fonction

Variable locale

Valeur retournée par la fonction